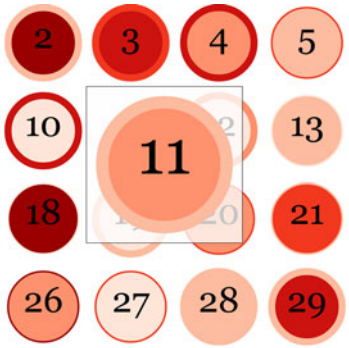


Figure 8.10 One of our gallery images in mid-transition. A border and background are added for UX purposes—the transparent regions of a PNG still register mouse2 events, and so the user should be reminded of the effective region for mouseover events.

didn't use array position to place the individual gallery images. This is why `redrawGallery` keeps drawing images in the right place, even after we start shifting images around in the DOM by mousing over them.

When you run the code in listing 8.11, D3's transitions are smart enough to process the `rgba` string designating a transparent background, as shown in figure 8.10. In some cases, like the next example, you may have to use D3's tweening capabilities to make sure that a DOM element interpolates properly. It probably doesn't follow the rules that make shape and color transitions work so easily. Still, with color and simple size transitions, you can use exactly the same code for `<div>` elements that we used with `<rect>` and `<circle>` elements, unless you're trying to transition to `"height:auto"` or some other nonnumerical value.

8.4.2 Selecting

Our final example adds a drop-down list to select a particular image and scroll the gallery to the row that holds that image. To do so, we need to populate the `<select>` element with choices that correspond to our images, and write a function that scrolls the gallery to the correct line. If you know how a `<select>` element works (it has a bunch of `<option>` elements nested underneath it in the DOM), you should guess how to do this with D3 using `imageArray` as your data. But creating the scrolling function is a bit more involved because we need to write a custom tween to scroll the `<div>` element that contains our gallery.

Listing 8.12 Zoom to a specific image from a select input

```
function zoomTo() {
  var selectValue = d3.select("select").node().value;
  var newWidth = parseFloat(d3.select("div.gallery")
                            .node().clientWidth);
  var imageSize = newWidth / 8;
  var scrollTarget = Math.floor(selectValue / 8) * imageSize;

  d3.selectAll("img")
       .filter(function(d) { return d.x == selectValue; })
       .transition().duration(2000).style("width", imageSize * 2)
```

Gets the selected image value

Calculates where that image is